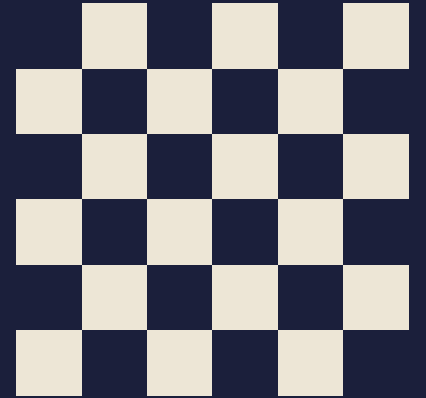


FUNDAMENTOS DE LA PROGRAMACIÓN

Bloque 3

Estructuras de datos



Objetivos del bloque



Agrupar datos relacionados

Usar listas, tuplas y diccionarios para guardar conjuntos de valores en una sola variable, en vez de una variable por dato.



Elegir la estructura correcta

Reconocer cuándo conviene una lista, una tupla o un diccionario según lo que se necesita hacer con los datos.

Contenidos

1

3.1 Listas: crear, indexar y trocear

2

3.2 Métodos de listas más usados

3

3.3 Listas anidadas: matrices

4

3.4 Tuplas

5

3.5 Diccionarios

6

3.6 Recorrer diccionarios con for

7

3.7 ¿Lista de listas o diccionario?

8

3.8 Errores comunes: IndexError y KeyError

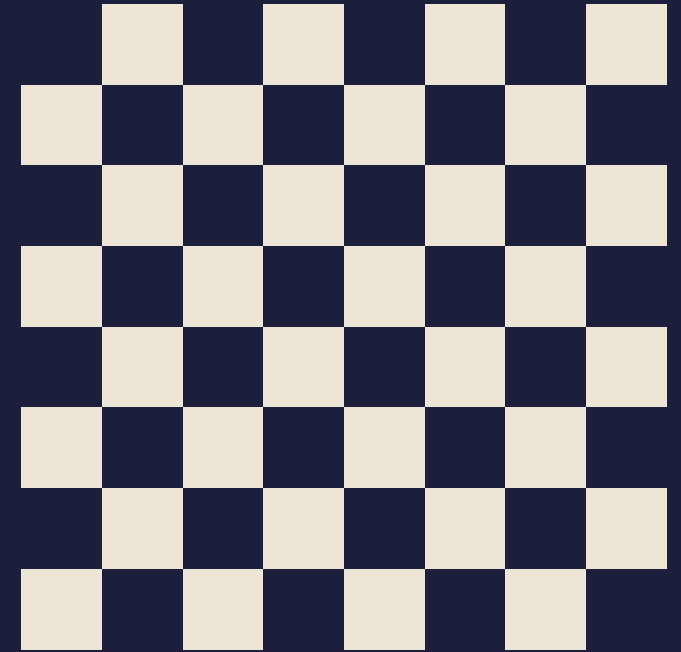
Cierre · ¡Empieza la Entrega 1 del proyecto!

¿RECUERDAS ESTO?

¿Y si quisiéramos guardar
no solo el color de cada
casilla, sino qué pieza hay?

```
casilla_a1: str = "torre_blanca"  
casilla_b1: str = "caballo_blanco"  
# ... y así, 64 veces. No, gracias.
```

En este bloque aprendemos a guardar las 64 casillas — con su pieza o sin ella — en UNA sola variable.



¿Qué es una lista?

Una lista guarda varios valores, en orden, dentro de UNA sola variable. Se escribe entre corchetes [], separando los elementos con comas.

```
letras_columnas: list = ["a", "b", "c", "d", "e", "f", "g", "h"]  
print(letras_columnas)  # ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

Visualmente, una lista es como una fila de casillas numeradas, empezando en 0 — el mismo "empezar en 0" que ya conoces de las cadenas.



Crear y acceder a una lista

```
letras_columnas: list = ["a", "b", "c", "d", "e", "f", "g", "h"]

letras_columnas[0]      # 'a'      (primer elemento)
letras_columnas[7]      # 'h'      (último elemento)
letras_columnas[-1]     # 'h'      (también el último)
len(letras_columnas)    # 8        (cuántos elementos hay)
letras_columnas[0] = "z" # ¡sí se puede! (a diferencia de un str)
```

Las listas **SÍ** se pueden modificar (son mutables)

A diferencia de las cadenas (str), que son inmutables, en una lista puedes cambiar un elemento directamente por su índice:

`letras_columnas[0] = "z"` es válido y cambia la lista de verdad.

Slicing: trocear una lista

Igual que con las cadenas, [inicio:fin] extrae una parte de la lista (inicio incluido, fin excluido):

```
letras_columnas: list = ["a", "b", "c", "d", "e", "f", "g", "h"]

letras_columnas[0:3]    # ['a', 'b', 'c']
letras_columnas[:3]     # ['a', 'b', 'c']    (igual, sin inicio = desde el 0)
letras_columnas[5:]     # ['f', 'g', 'h']    (sin fin = hasta el final)
letras_columnas[:]      # copia completa de la lista
```

El resultado de trocear una lista es SIEMPRE otra lista (aunque tenga un solo elemento).

Ahora te toca a ti

Las piezas capturadas (*≈ 6 minutos*)

Declara una lista capturadas: list con tres nombres de piezas (str). Imprime la primera y la última pieza capturada, cuántas hay en total, y las dos últimas usando slicing.

```
capturadas: list = ["peón", "caballo", "alfil"]  
# completa con print()...
```

Pista

"las dos últimas" con slicing es capturadas[-2:] — piensa qué significa "desde la posición -2 hasta el final".

Métodos de listas: añadir y quitar

```
capturadas: list = []

capturadas.append("peón")      # añade al final
capturadas.append("alfil")     # ['peón', 'alfil']

capturadas.insert(0, "dama")   # inserta en la posición 0
# ['dama', 'peón', 'alfil']

capturadas.remove("peón")      # quita por VALOR (el primero que encuentra)
ultima: str = capturadas.pop() # quita y devuelve el ÚLTIMO elemento
```

append() añade al final · insert() añade en una posición concreta · remove() quita por valor · pop() quita por posición (y te devuelve lo que quitó).

Métodos de listas: ordenar y buscar

```
valores: list = [3, 9, 1, 5]

valores.sort()           # [1, 3, 5, 9]   (ordena de menor a mayor)
valores.reverse()        # [9, 5, 3, 1]   (invierte el orden)

valores.index(5)          # 1           (posición del primer 5)
valores.count(5)          # 1           (cuántas veces aparece)
5 in valores              # True        (¿está el valor en la lista?)
```

sort() y reverse() modifican la lista original (no crean una nueva ni la muestran: hay que hacer print(valores) después para verla).

La trampa: `append()` no es lo mismo que `+`

Antes de ejecutar: ¿qué imprime cada una de estas dos líneas?

```
lista_a: list = [1, 2, 3]
lista_a.append([4, 5])
print(lista_a)  # [1, 2, 3, [4, 5]] ← ¡una lista DENTRO de la lista!

lista_b: list = [1, 2, 3]
lista_b = lista_b + [4, 5]
print(lista_b)  # [1, 2, 3, 4, 5]
```

`append()` añade el objeto TAL CUAL, aunque sea una lista

`append([4, 5])` añade la lista `[4, 5]` como UN ÚNICO elemento nuevo, no sus dos valores por separado.

Si quieres "pegar" los elementos de una lista a otra, usa `+` (crea una lista nueva) o el método **`extend()`** (modifica la original añadiendo los elementos uno a uno).

Listas anidadas: una lista de listas

Un elemento de una lista puede ser, a su vez, otra lista. Así se representan tablas y matrices: una lista "externa" de filas, donde cada fila es una lista de valores.

```
matriz: list = [  
    [1, 2, 3], # fila 0  
    [4, 5, 6], # fila 1  
    [7, 8, 9], # fila 2  
]
```

3 filas × 3 columnas = una matriz 3×3, exactamente como una hoja de cálculo — o un tablero de ajedrez.

Indexar una matriz: fila y columna

Para acceder a un valor concreto, se usan DOS índices seguidos: [fila][columna].

```
matriz: list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

matriz[0]          # [1, 2, 3]      (toda la fila 0)
matriz[0][0]       # 1              (fila 0, columna 0)
matriz[1][2]       # 6              (fila 1, columna 2)
matriz[2][0] = 99  # cambia el 7 por un 99
```

El primer índice es SIEMPRE la fila, el segundo SIEMPRE la columna — igual que en el tablero (fila, columna) que ya usabais desde el Bloque 1.

Aplicación directa: el tablero como matriz

```
tablero: list = []

for fila in range(8):
    fila_vacia: list = ["."] * 8    # 8 casillas vacías
    tablero.append(fila_vacia)

# Colocar el rey blanco en e1 (fila 0, columna 4):
tablero[0][4] = "\u2654"

print(tablero[0][4])    # ♔
```

Esto es, literalmente, una de las dos formas válidas de guardar el tablero completo en la Entrega 1.

Ahora te toca a ti

Coloca dos piezas (*≈ 7 minutos*)

Crea una matriz 8×8 de casillas vacías ("·"), como en el ejemplo. Coloca el rey negro en e8 (fila 7, columna 4) y el rey blanco en e1 (fila 0, columna 4). Imprime esas dos casillas para comprobarlo.

```
tablero: list = []  
# construye la matriz con un for, como en el ejemplo...
```

Pista

"e8" es la fila 7 (arriba del todo) y "e1" es la fila 0 (abajo del todo) — recuerda que las filas del tablero se numeran de abajo arriba, pero los índices de la lista van de 0 en adelante.

Tuplas

Una **tupla** se parece mucho a una lista — guarda varios valores en orden — pero se escribe con paréntesis () y, una vez creada, **no se puede modificar** (es inmutable, como los str).

```
posicion: tuple = (4, 2)    # (fila, columna) — dos valores que van juntos

posicion[0]                # 4    (se lee igual que una lista)
posicion[1]                # 2
posicion[0] = 9             # TypeError: 'tuple' object does not support...
```

Se leen igual que las listas (con []), pero no se pueden reasignar sus elementos una vez creadas.

Tuplas vs listas: ¿cuándo usar cada una?

Lista — []

Usa una lista cuando los datos pueden cambiar: añadir, quitar o modificar elementos (las piezas capturadas, el historial de movimientos...).

Tupla — ()

Usa una tupla cuando los datos forman un conjunto fijo que no debería cambiar (una posición (fila, columna), un color RGB (255, 0, 0)...).

Regla rápida: si en algún momento vas a escribir `.append()` o vas a reasignar un elemento, necesitas una lista, no una tupla.

¿Qué es un diccionario?

Un diccionario guarda pares clave → valor, entre llaves { }. En vez de buscar por posición (0, 1, 2...) como en una lista, buscas por una clave con significado propio.

```
pieza_en: dict = {  
    "e1": "rey_blanco",  
    "e8": "rey_negro",  
}  
  
pieza_en["e1"] # 'rey_blanco' (accedes por la clave, no por posición)
```

Es exactamente la otra forma válida de guardar el tablero en la Entrega 1: la clave es la casilla ("e4"), el valor es la pieza.

Añadir, modificar y eliminar claves

```
pieza_en: dict = {}

pieza_en["e1"] = "rey_blanco"    # añade la clave "e1"
pieza_en["e1"] = "dama_blanca"  # si ya existía, la SOBREScribe

del pieza_en["e1"]              # elimina la clave (y su valor)

"e1" in pieza_en                # True / False – ¿existe esa clave?
```

Cada clave es única: si asignas dos veces a la misma clave, la segunda sustituye a la primera (no se duplica).

El método `.get()`: evitar errores

Antes de ejecutar: ¿qué pasa si preguntas por una clave que NO existe?

```
pieza_en: dict = {"e1": "rey_blanco"}  
print(pieza_en["e4"]) # KeyError: 'e4'
```

`get()` devuelve un valor por defecto en vez de fallar

`pieza_en.get("e4", "vacío")` → devuelve "vacío" si "e4" no existe, en vez de interrumpir el programa con un error.

Es exactamente lo que necesitas para las casillas vacías del tablero: en vez de comprobar antes si `"e4" in pieza_en`, puedes escribir directamente:

```
contenido: str = pieza_en.get("e4", "Casilla vacía")
```

Recorrer un diccionario con for

```
pieza_en: dict = {"e1": "rey_blanco", "e8": "rey_negro"}

for casilla in pieza_en:
    print(casilla)          # solo las claves: e1, e8

for pieza in pieza_en.values():
    print(pieza)           # solo los valores: rey_blanco, rey_negro

for casilla, pieza in pieza_en.items():
    print(casilla, "->", pieza) # e1 -> rey_blanco, e8 -> rey_negro
```

Ahora te toca a ti

Diccionario de valores (*≈ 6 minutos*)

Crea un diccionario `valor_pieza` con la clave/valor de cada tipo de pieza (peón:1, caballo:3, alfil:3, torre:5, dama:9). Con un `for` y `.items()`, imprime cada pieza junto a su valor.

```
valor_pieza: dict = {"peon": 1, ...}  
# completa con for + .items()...
```

Reto extra

Usando el rey con valor 0 (por convención no se "captura" nunca), calcula la suma total de valor de todas las piezas de un bando con un acumulador y un `for`.

¿Lista de listas o diccionario? Para el tablero

Lista de listas

```
tablero[filas][columnas]
```

Natural para recorrer TODO el tablero con bucles anidados (fila por fila, columna por columna) — como ya hicisteis en el Bloque 2.

Diccionario

```
pieza_en["e4"]
```

Natural para consultar UNA casilla concreta por su nombre, y para no tener que guardar las 64 casillas vacías (solo las que tienen pieza).

Para la Entrega 1 vale cualquiera de las dos — elige la que mejor entiendas: no hay una "correcta", solo distintas ventajas según qué operación hagas más.

Error común: IndexError

Antes de ejecutar: ¿qué crees que pasará?

```
letras_columnas: list = ["a", "b", "c", "d", "e", "f", "g", "h"]  
print(letras_columnas[8])    # ???
```

IndexError: list index out of range

La lista tiene 8 elementos, en las posiciones 0 a 7. La posición 8 no existe — es el mismo "off-by-one" del Bloque 2 (range llega hasta n-1), pero ahora aplicado a listas.

Es un error MUY habitual al trabajar con el tablero: `columna = 8` (pensando "la octava columna") en realidad es la NOVENA posición, que no existe. Las columnas válidas son 0 a 7.

Error común: KeyError

Antes de ejecutar: ¿qué crees que pasará?

```
pieza_en: dict = {"e1": "rey_blanco"}  
print(pieza_en["z9"]) # ???
```

KeyError: 'z9'

"z9" no es una clave del diccionario (además de no ser ni siquiera una casilla real del tablero). Acceder con [] a una clave que no existe interrumpe el programa.

La solución, como viste antes: usa `.get("z9", valor_por_defecto)` en vez de `["z9"]` cuando la clave pueda no existir.

Resumen del Bloque 3

- ✓ Crear listas y acceder a sus elementos por índice
- ✓ Modificar listas: append, insert, remove, pop
- ✓ Usar listas anidadas para representar matrices
- ✓ Crear y usar tuplas, y saber cuándo preferirlas
- ✓ Usar `.get()` para evitar errores con claves inexistentes
- ✓ Reconocer `IndexError` y `KeyError`
- ✓ Trocear listas con slicing
- ✓ Ordenar y buscar en listas: `sort`, `index`, `count`, `in`
- ✓ Indexar una matriz con `[fila][columna]`
- ✓ Crear diccionarios y acceder por clave
- ✓ Recorrer diccionarios con `.items()`, `.keys()`, `.values()`

Con esto ya tienes todo lo necesario para la Entrega 1: guardar el tablero completo con las 32 piezas, en una única estructura de datos.

PRÓXIMO PASO

¡Empieza la Entrega 1!

El tablero con piezas: la primera entrega evaluable del proyecto. Todo lo que necesitas ya lo tienes — variables, control de flujo, y ahora, estructuras de datos.